Practical Exercise 1: AWS Computing Services

In this exercise, we will explore a foundational service in the AWS platform, namely, Amazon EC2 (i.e. Amazon Elastic Compute Cloud).

To start building atop the AWS platform as part of these exercises, you need to ensure that you met the following requirements:

1. You signed up for AWS Academy platform.
2. You have access to your AWS Management Console environment provided to you via the AWS Academy platform.
3. You have downloaded a security keypair associated with your current AWS Academy lab session.
4. You have created your security credentials.

What you are expected to learn from this exercise

- Identify and understand many of the available AWS Cloud Computing services and their applications.
- Understand how load balancing and auto scaling can add value to a cloud application.
- Create and use a VM on AWS from scratch.
- Deploy a client-server application using VMs and enable communication between them.

Activity 1: Create EC2 Instance

Follow the instructions below to create an EC2 instance using the AWS Management Console.

Step 1: Go to your AWS console.

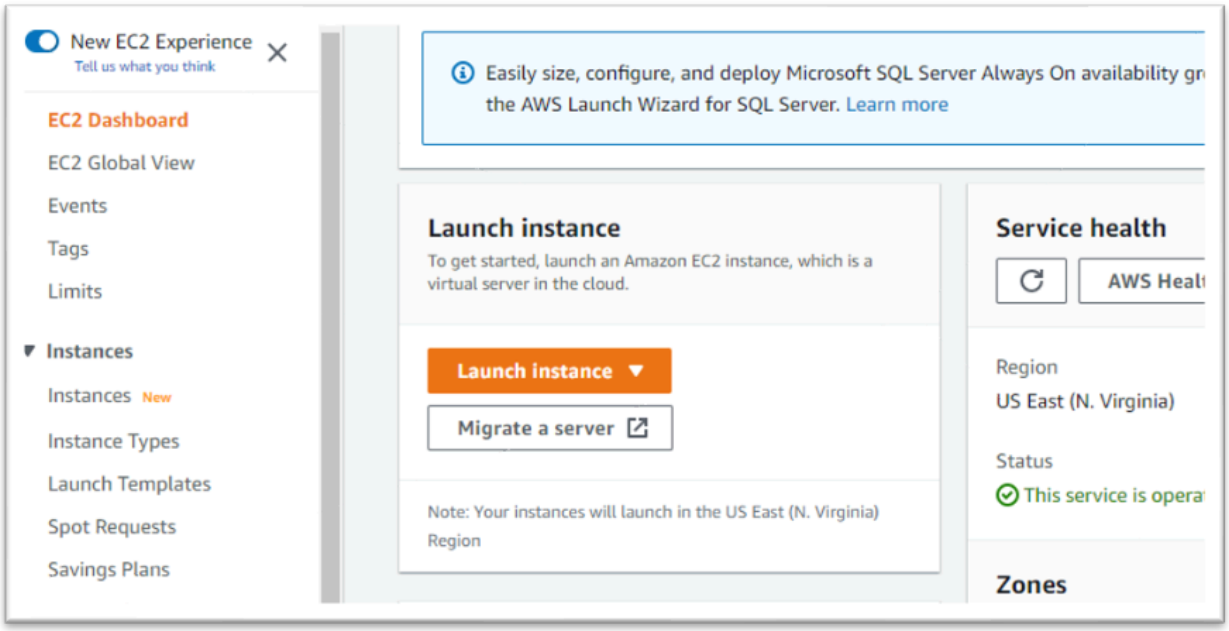
Make sure that you use **US East (N. Virginia)** `us-east-1`**,** or **US West (Oregon)** `us-west-2` as your region.

Note: AWS Academy only allows you to create EC2 instances (or any other AWS resource) within the two AWS regions.

Step 2: Click the **All Services** menu.

Step 3: Select **EC2** from the **Compute** section.

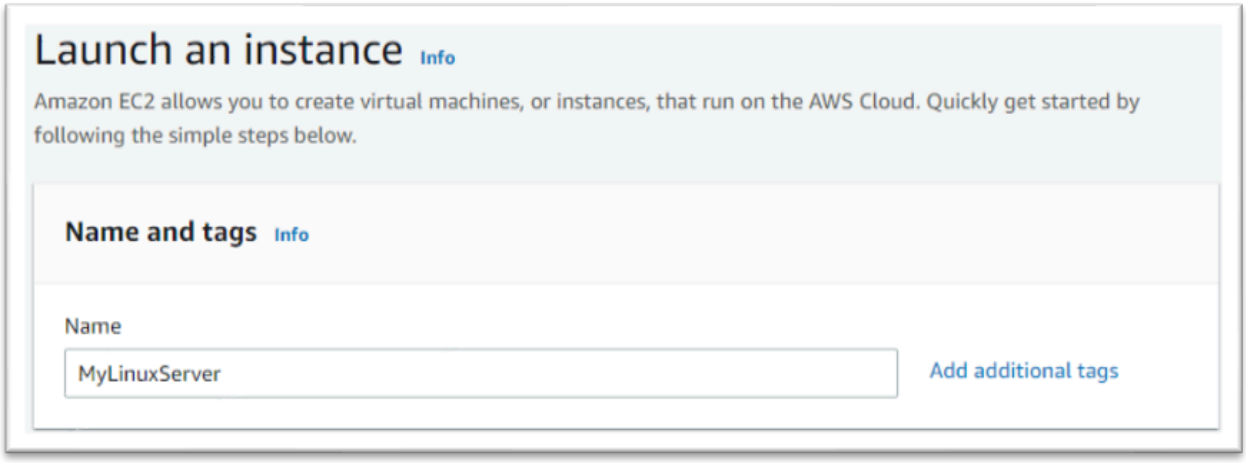
Step 4: From the next page, click **Launch Instance**. In this example we will launch and configure a virtual machine running **Ubuntu Server 24.04 (64-bit)** operating system.



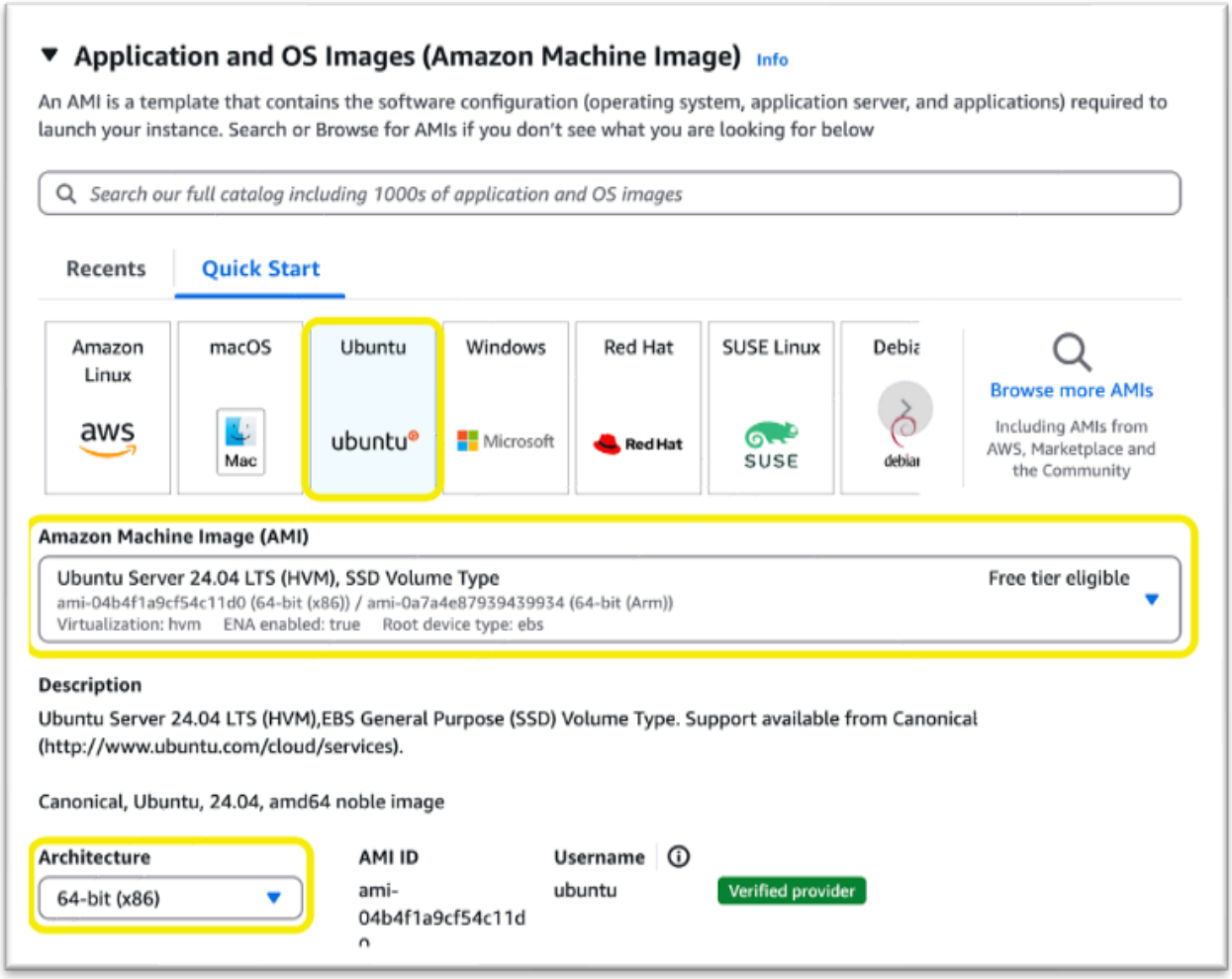
Activity 2: Create a Linux Virtual Machine (VM)

Note: In this task, we will focus only on creating a VM running **Ubuntu Server 24.04 (64-bit).**

Step 1: Choose a name you prefer for your instance and type it into the Name field in **Name and tags** section. We named it "**MyLinuxServer**".

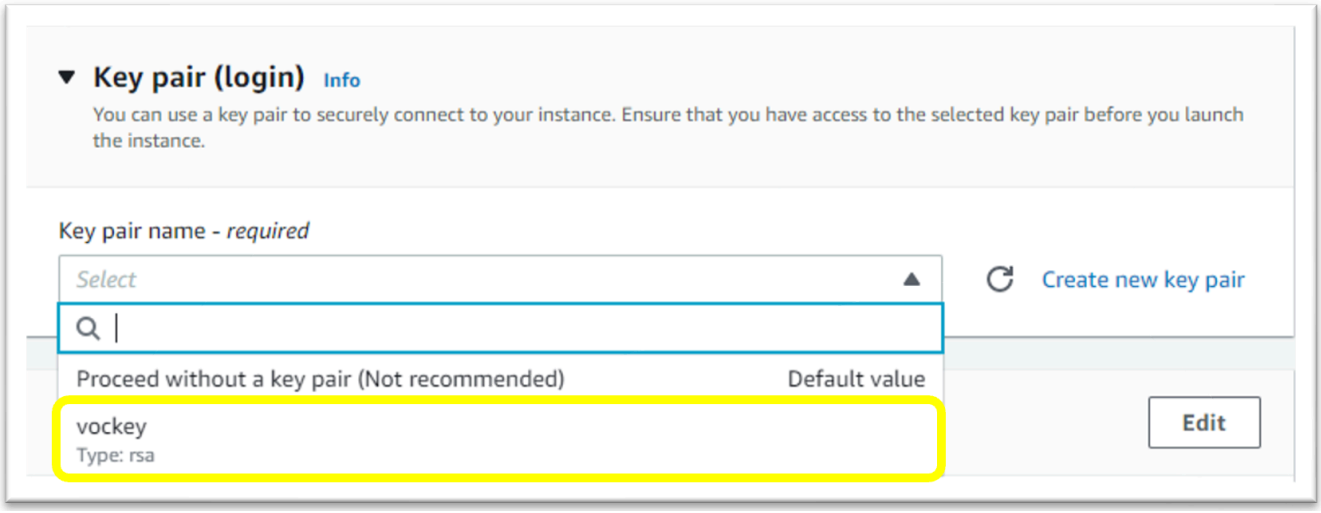


Step 2: From the **Application and OS Images** section, select **Ubuntu** from the Quick Start tiles listed. Expand the drop-down list **Amazon Machine Image (AMI)** and select **Ubuntu Server 24.04 LTS**. Leave the Architecture sub-section with its default value (i.e. 64-bit).



Step 3: Select t3.micro from the **Instance Type** section, as below. This is also what is selected by default.

Step 4: When you are in the **Key pair (login)** section, expand the **Key pair name** drop down box. Out of what is listed, select **vockey** as your preferred key pair. This is what you will use to later log in to your EC2 instance via SSH.



The pre-provided SSH key pair ("**vockey**") refers to the default key pair created for the AWS Academy lab session you have previously initialised. **Vockey works only in `us-east-1`.** If you launch your EC2 in **us-west-2**, create a new key pair in **us-west-2** and download it, otherwise you won't be able to SSH into the instance.

To download it, go to your AWS Academy portal through which you launched your current session, and select AWS Details, as below.

Then, download the appropriate key file that matches the operating system you will later connect to this virtual machine from. For instance, if you are connecting from Linux/MacOSX, click Download PEM. If you are on Windows, click Download **PPK**.

Step 5: Go to **Network Settings** section and click **Edit**. Here we will configure a Security Group, which is a similar concept to set firewall rules. Rename the security group with a suitable name (e.g.MyLinuxSecurityGroup). You can also add a description to your Security Group. Select, **Add security group rule** to open and configure the following ports:

- SSH (This should already be configured for you, by default)
- HTTP
- HTTPS
- All TCP
- All ICMP - IPv4

Please note that for this task, we select source Anywhere as the Source type for all. This allows external access to your EC2 instance(s) that are configured with the security group configured above from any arbitrary location. This widens the attack surface associated with your EC2 instance and can make it open to unauthorized access. **In real-life, however, you would prefer to configure it**

as either custom IP or My IP for enhanced security. For example, if you select My IP for SSH, then you will be able to SSH only from your current machine.

After adding the ports, the **Network Settings** configuration will look like the following:

Step 6: Leave all other configurations in their defaults.

Step 7: Review your choices, and then click "**Launch instance**".

Step 8: Next in your AWS console you will see a status page, notifying you that your instances are launching. Click "**Instances**" to continue after your instances have been properly launched.

Step 9: Your instance will be displayed in the list of running EC2 instances. If the instance state is *not running*, wait a few moments and press the rounded arrow icon to refresh the list. When it is **running**, continue to the next step.

Step 10: Select the instance to display the instance details. Identify the **Public DNS** value and copy it to the clipboard.

Activity 3: Connect to your VM Instance

Follow the instructions below to connect to the EC2 VM you have created earlier. Make sure you follow the correct steps that are relevant for your OS.

Option 1: Mac/Linux Users ONLY

If you are using Linux or MacOS, the built-in **SSH** client should allow you to connect to your EC2 instance directly from a terminal/command window.

Step 1: To connect with your virtual server, open the terminal window and type the following command.

```
ssh -i [path_to_your_key_file]/[your_key_file_name].pem u
buntu@[your_instance_public_DNS]
```

For example:

```
ssh -i /Users/[your_login_name]/Documents/Keys/mykeypair.
pem ubuntu@ec2-54-68-87-178.us-west-2.compute.amazonaws.c
om
```

Step 2: Please provide the location and name of your key file (.pem file) correctly have saved before. If any yes/no question appears type yes, then hit enter. Now you are connected to your Amazon instance. In Linux/MacOS, if you see an error that looks something like “Key is too open”, change the degree of file permissions of the key file by using the chmod command, as below:

```
chmod 400 [path_to_your_key_file]/[your_key_file_name].pe
m
```

For example:

```
chmod 400 /usr/home/myLinuxKey.pem
```

And then try again to connect. If you still see any errors, please contact your tutor.

Option 2: Windows Users ONLY

If you are using Windows, you will need PuTTY to connect to your virtual machine via SSH. If you don’t have it already installed, please download it from **https://www.putty.org/.**

Step 1: Your private key is the **.ppk file** you downloaded before. which you will now use with PuTTY. You can now connect to your instance using PuTTY's SSH client.

Step 2: Click on connection and put 300 in Seconds between keep alive as in the following screen.

Step 3: Now click on **Session**.

In the Host Name box, enter **ubuntu@[your_instance_public_DNS]** (example: **ubuntu@ec2-54-68-87-178.us-west-2.compute.amazonaws.com**).

Ensure that Port is 22 and Connection type is SSH.

Step 4: Now in the **Category** pane, expand **Connection**, expand **SSH**, expand **Auth**, and then select **Credentials**. Click **Browse** and select the **.ppk** file you downloaded previously, and then click **Open**.

Step 5: If you plan to start this session again later, you can save the session information for future use. Select **Session** in the Category tree, enter a name for the session in Saved Sessions, and then click **Save**.

Step 6: Click **Open** to start the PuTTY session.

Step 7: PuTTY displays a security alert dialog box that asks whether you trust the host you are connecting to.

Step 8: Click **Accept**.

A window opens and you are connected to your instance.

If you have any issues with any of the instructions above, or if your VM isn't working, please feel free to contact your tutor for help.

Activity 4: Customise your VM Instance

AWS provides the required deployment infrastructure to run your applications. That is, in this scenario, the VM you created previously is a basic unit of deployment that AWS offers to run your applications. What should happen if you need to run an application atop the VM? For instance, imagine you want to **run a Java application** in this VM. To facilitate that, you would want to have the Java Runtime Environment (JRE) installed. Moreover, you might also want to use this VM as a hosting server so you can **deploy your application in a web server environment such as Apache2 server**.

Follow the instructions below to customise your VM instance to achieve your goals.

Install JDK into your VM Instance

1. Update your package list to ensure you have the latest information:

```
sudo apt update

sudo apt upgrade
```

2. You can install the OpenJDK version of the Java SDK. For example, to install OpenJDK 17, run:

```
sudo apt install openjdk-17-jdk
```

3. Check the Java is installed correctly by running:

```
java -version
```

   It should show Java “17.0.18” as default version.

4. To confirm the location of the Java compile (javac), run:

```
which javac
```

   This should return a path like `/usr/bin/javac`. Now you can run any executable .jar file or .class file in the virtual machine.

Install Apache2 Server into your VM Instance

1. Install Apache server using the following command.

```
sudo apt-get install apache2
```

2. Install PHP using the following commands.

```
sudo apt-get install php libapache2-mod-php
```

3. Verify your PHP installation:

```
php -v
```

4. Enable the correct modules accordingly (as PHP 8.3 is installed):

```
sudo a2enmod mpm_prefork && sudo a2enmod php8.3
```

5. Start the Apache web server using the following command:

```
sudo service apache2 start
```

6. If you see Apache server already started and running before, then restart it by running the following command:

```
sudo service apache2 restart
```

7. Test your web server environment. In a web browser, enter the **public IPv4 DNS address (or the public IPv4 address)** of your instance. You can get the Public IPv4 DNS address for your instance using the Amazon EC2 console. as below.

8. You should now see the Apache server test page **via http** (**instead of https**).

Activity 5: Using Apache Server

Follow the steps below to learn how to use the Apache server to host a simple PHP application.

Step 1: Create a file named **index.php**, which prints “Hello World!” when invoked.

```
<?php echo "Hello World!"?>
```

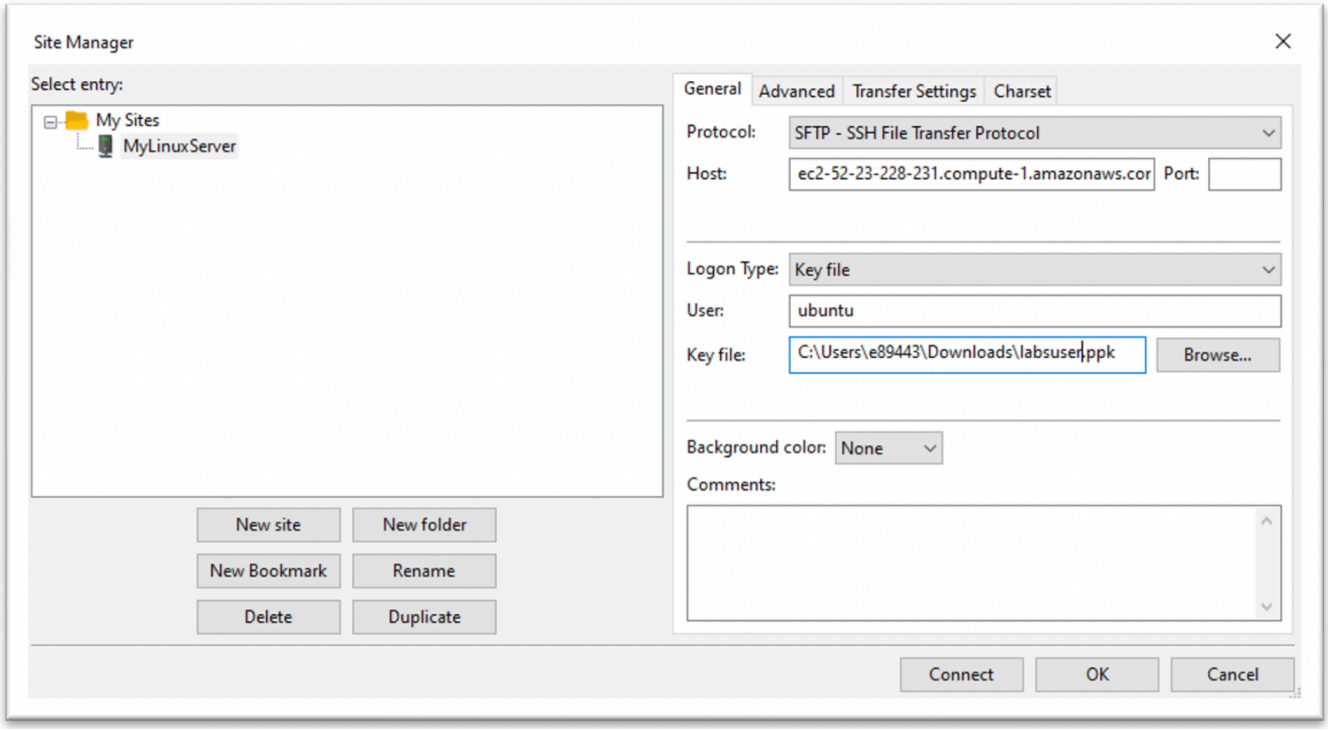
Step 2: Go back to the console of your **MyLinuxServer** and set permissions of your web root folder using the command:

```
sudo chmod -R 777 /var/www/html
```

Step 3: Now upload the **index.php** you have previously created to `/var/www/html/` of your MyLinuxServer EC2 instance using your preferred tool.

To upload files to your VM, follow the steps below:

1. Open your preferred remote file sharing client (i.e. Filezilla, WinFCP, SCP, etc) if you already have it in your computer (otherwise download or install it).
2. If you choose to use **Filezilla**, click **File>Site Manager>New Site**. Give an appropriate name (i.e. **MyLinuxServer)**.
3. Configure **Host** with the **Public IPv4 Address** of your EC2 instance, **Port** as 22, and Protocol as SFTP.
4. Select **Logon Type** as “Key file” and type “ubuntu” as the **User**. Now click the browse button and select the .ppk or .pem file you have downloaded previously.



You should now see the home directory of the default Ubuntu user that you are connecting to your EC2 instance from.

Now upload the **index.php** you have previously created to `/var/www/html/` of your MyLinuxServer EC2 instance.

Step 4: Remove the existing **index.html** file from `/var/www/html` directory.

Step 5: Browse the Public IPv4 DNS address of **MyLinuxServer** instance again using your web browser. Now you should see a simple PHP page we came up with earlier in action instead of Apache Server test page we had seen previously.

Activity 6: Deploy a Client-Server Application with VMs

The purpose of this activity is to teach you how to easily deploy a client-server application using virtual machines. We will learn how to duplicate and configure VM instances for such a scenario without having to create new EC2 instances from the scratch.

This activity will take you through four tasks that will help you to prepare the deployment environment to run a simple client-server application from uploading files, to duplicating a VM and then enabling client-server communication between the two VMs.

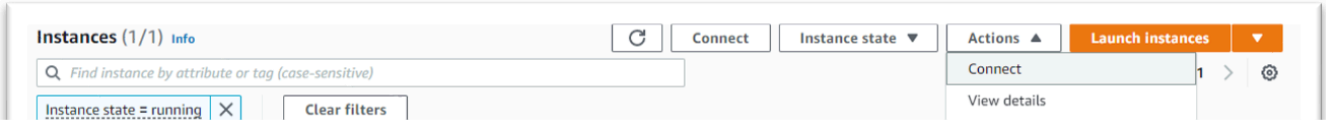
Step 1: Upload files into your VM

Download the file **Server.java** to your local machine. Then upload it into your VM working directory (for example, your home directory `/home/ubuntu`) by following the instructions in Step 3 of Activity 5.

Step 2: Duplicate your EC2 Instance

Using the steps below to duplicate your EC2 instance created previously.

1. Select the instance that we just created (e.g. MyLinuxServer) from AWS EC2 console (as in the figure). Then Select **Actions > Image and templates > Launch more like this**.



2. In the next step, go to the **Name and tags** section, and configure the name of this instance as **MyLinuxClient.**

3. Go to the **Key pair (login)**section and select the same key pair (e.g. vockey) you have previously configured for the **MyLinuxServer** instance**.**
4. Then click **Launch instance** to launch your duplicate EC2 instance.
5. Open another PuTTY console/terminal, connect to **MyLinuxClient** using the same process as before, and install OpenJDK 17 by following the same process described in [Activity 4](#).

Step 3: Enable client-server communication between the two VMs

Follow the steps below to enable client-server communication between the two VMs you have previously created.

1. Download the file **Client.java** to your local machine.

   Open the file using your preferred text editor and modify the hostname configured for the client-side socket **localhost** to **public IPv4 DNS address** of your **MyLinuxServer**.

2. Upload the **modified** **Client.java** file to **MyLinuxClient** into your VM working directory (for example, your home directory `/home/ubuntu`).

   For this, you will have to create another site that facilitates connectivity to your newly created EC2 instance – **MyLinuxClient**. We have covered this previously.

3. Now go back to PuTTY console/terminal of **MyLinuxServer**. Compile **Server**.java:

```
javac Server.java
```

4. Move to PuTTY console/terminal of Compile **Client**.java:

```
javac Client.java
```

5. Go back to PuTTY console/terminal of **MyLinuxServer**.

   Run the **Server**:

```
java Server
```

   You will see the message Server is now running at port: 11113

6. Go back to PuTTY Console/terminal of **MyLinuxClient.**

   Run the **Client**:

```
java Client
```

   You will find your two VMs are communicating using TCP protocol.

7. Now you can stop of your **MyLinuxServer** and **MyLinuxClient, and End Lab.**